

1. Derive the conditional VAE formula

Suppose our model is θ , prior p , posterior q and condition c . In inference, on an observation x , we sample a vector z obeys the prior and we want to optimize the likelihood $p(x|c; \theta)$. However, if we do

$$p(x|c; \theta) = \int_Z p(x, z|c; \theta) dz,$$

directly, the integration is often intractable. So we'll introduce the posterior $q(z|x, c; \theta')$ and we'll maximize a lower bound (ELBO) in following derivation. Further, instead of optimizing $p(x|c; \theta)$ directly, we optimize its log value for using Jensen's inequality to derive a lower bound. So we have

$$\begin{aligned} & \log p(x|c; \theta), \text{ by joint distribution formula} \\ &= \log \int_Z p(x, z|c; \theta) dz \\ &= \log \int_Z q(z|x, c; \theta') \frac{p(x, z|c; \theta)}{q(z|x, c; \theta')} dz \\ & \quad , \text{ since log is concave, by Jensen's inequality} \\ &\geq \int_Z q(z|x, c; \theta') \log \frac{p(x, z|c; \theta)}{q(z|x, c; \theta')} dz \\ & \quad , \text{ again by joint distribution formula} \\ &= \int_Z q(z|x, c; \theta') \log \frac{p(x|z, c; \theta)p(z|c; \theta)}{q(z|x, c; \theta')} dz \\ &= \int_Z q(z|x, c; \theta') \log p(x|z, c; \theta) dz \\ & \quad + \int_Z q(z|x, c; \theta') \log \frac{p(z|c; \theta)}{q(z|x, c; \theta')} dz \\ & \quad , \text{ by definition of expectation} \\ &= \mathbb{E}_{z \sim q(z|x, c; \theta')} (\log p(x|z, c; \theta)) \\ & \quad - \int_Z q(z|x, c; \theta') \log \frac{q(z|x, c; \theta')}{p(z|c; \theta)} dz \\ & \quad , \text{ by definition of KL divergence} \\ &= \underbrace{\mathbb{E}_{z \sim q(z|x, c; \theta')} (\log p(x|z, c; \theta))}_{\text{the reconstruction quality}} - \underbrace{\text{KL}(q(z|x, c; \theta') || p(z|c; \theta))}_{\text{latent space regularization}} \end{aligned}$$

So we maximize the ELBO, which is the sum of reconstruction quality and latent space regularization, hoping to reach the maximum (log-)likelihood of our $p(x|c; \theta)$.

2. Introduction

In this task, we aim to implement a Conditional Variational Autoencoder (C-VAE), which is a type of VAE conditioned on additional information, for video prediction based on the first frame of a video and the pose in every frame as labels of C-VAE.

3. Implement details

A. Implement Video prediction protocol in training/testing stage

Let's talk about the protocol in training first.

- In training, for a 16 frames of image $(X_1, \dots, X_{16})$ and pose $(P_1, \dots, P_{16})$, in predicting a frame $\widehat{X}_i$, we first take X_i, P_i to predict a distribution in the latent space, then draw a sample z from that. This is the encoder part.
For the decoder part, we feed model X_{i-1}, P_i and z (with teacher forcing)/ $\widehat{X}_{i-1}, P_i$ and z (without teacher forcing).
We have the following pseudo-code.

```
predicted_frame 1 = frame 1
initiate loss
Loop: frame i = 2..16
    mean, logvar = encoder(frame i, pose i)
    var = exp(logvar)
    sample z obeying N(mean, var)
    if i = 2 or teacher_forcing:
        input_frame = frame i
    else input_frame = predicted_frame i-1
    predicted_frame i = decoder(input_frame, pose i, z)
    loss += loss_image(predicted_frame i, frame i) + KL(N(mean,
        var), N(0, 1))
```

The N in "N(mean, var)" above denotes the normal distribution. Since the predicted frame in early epochs may not be in the valid dynamic range $(0 \sim 1)$. So we will clip the predicted_frame to the range

- In testing, the protocol is similar to the decoder part of training phase. However we will sample the latent vector z from the standard normal distribution $N(0,1)$ instead. The pseudo-code thus will be

```
predicted_frame 1 = frame 1
Loop: frame i = 2..video_length
    sample z obeying N(0, 1)
    if i = 2:
```

```

        input_frame = frame i
    else input_frame = predicted_frame i-1
    predicted_frame i = decoder(input_frame, pose i, z)

```

Practically, the encoder above in this task consists of three component: the frame encoder, the pose encoder and the posterior generator. And the decoder consists of the decoder fusion and the generator.

B. Implement reparameterization tricks

So after we find out the mean μ and variance σ^2 of our posterior, we sample z from $N(0,1)$ and then relocate it as if we sample it from $N(\mu, \sigma^2)$. Practically we do this

```

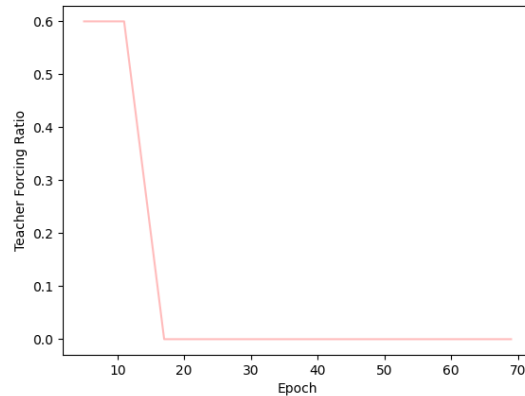
func reparameterize(mu, logvar):
    sd = exp(0.5*logvar)
    sample epsilon obeying N(0, 1)
    z = mu + sd * epsilon
    return z

```

That is, we extract the randomness of $z \sim N(\mu, \sigma^2)$ to ϵ . Then we remove the randomness outside the learning paradigm to help the effective back-propagation.

C. teacher forcing strategy

In one epoch, for a piece of training frames, we have a specific probability (training force ratio) to apply teacher forcing. The probability is initially set to 0.6 in the first 10 epoch, then in the next epoch, 0.5 so on and then 0 after reaching 0.



Teacher forcing ratio

D. kl annealing ratio

For KL annealing, there are three strategies: monotone, cyclical and none. This is about the KL divergence coefficient β in the loss function

$$\text{total loss} = \text{reconstruction loss} + \beta \times \text{posterior loss (KL divergence)}$$

- In monotonic strategy, our β increases linearly from 0 to 1 in first 5 epochs then stays at 1.
- In cyclical strategy, our β increases linearly from 0 to 1 in first 5 epochs then stays at 1 for next 5 epochs and then start again from 0.2 periodically.
- In none strategy, we don't do KL divergence annealing, leave the coefficient β at 1 in the whole training phase.

E. Strategy for the Kaggle/Other training strategy

The strategy used in the leaderboard is:

- Cyclical annealing with *beta* increases linearly from 0 to 1 in first 40 epochs then stays at 1 for next 10 epochs and then start again from 0 periodically.
- The tfr strategy is that tfr initially set to 0.6 in the first 10 epoch, then decrease by 0.1 for every epoch until reaching 0.
- Fast train is used, we use first 5 epochs training on 40% of the dataset.

To mitigate the KL vanishing issue, I tried a cyclical annealing strategy with longer period, expecting the model emphasize on building a preferable latent space. Maybe I ought to try a longer period after I analyse the KLD loss curve in the scenario above but I cannot due to the loong training time.

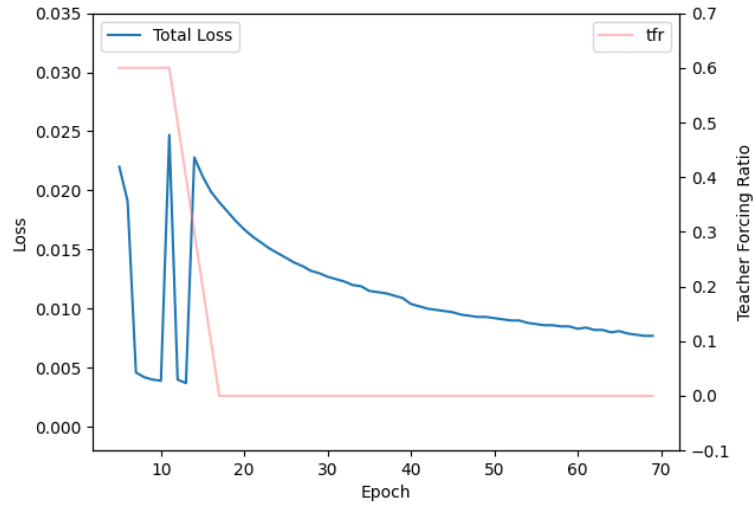
I decay the teacher forcing ratio so fast because I thought that we're inferencing on a 630-frames video, which is much longer than that in training phase. So I what the model give more emphasis on how to generate from its previous prediction.

After training, I pick a checkpoint with the lowest validation loss, which is at epoch 54, heuristically believe in its ability on unseen data.

4. Analysis & Discussion

A. Plot Teacher forcing ratio.

So this is the teacher forcing ratio compared to loss curve. It's a bit

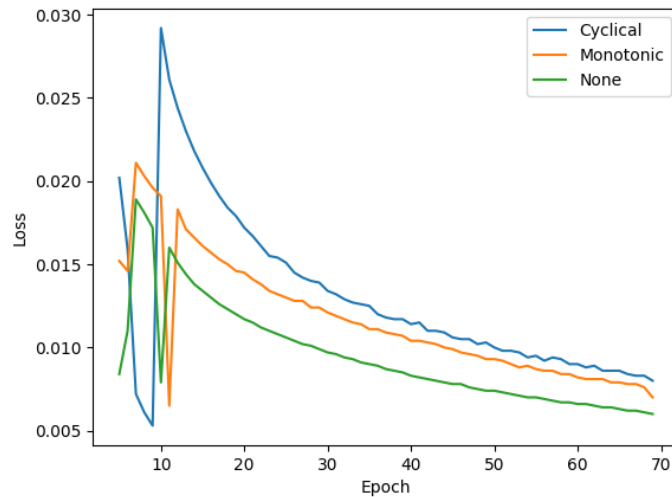


Tfr compared to loss curve

entangled in the early epochs in the figure. Basically the single-epoch loss is more based on whether the teacher forcing is applied on that epoch.

B. Plot the loss curve while training with different settings.

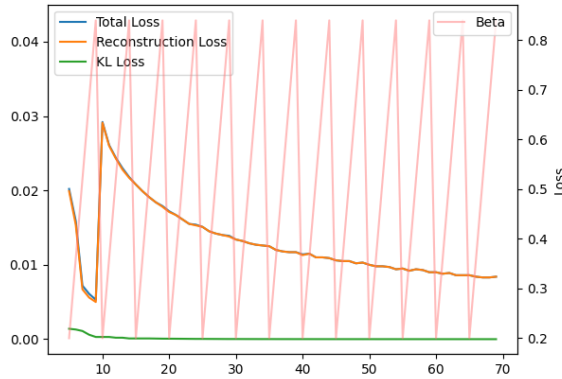
I set the fast train start for the first 5 epochs. Considered that we have different initiations of the model, we only plot the loss curve after the fast start.



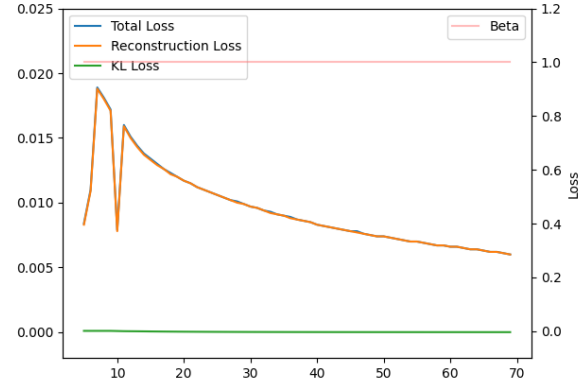
Loss curve in different scenarios

It can be weird that the none-strategy get better result in training loss. The first possible reason is that the model does not yet converge. Another reason may be that I encounter KL vanishing (so that the KL coefficient β does little on the loss).

I plot the components of loss here. We can see that the KL divergence

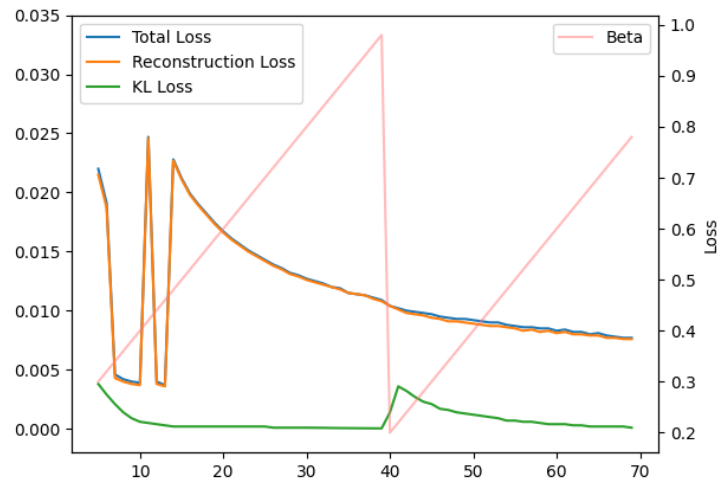


Cyclical



None

vanished immediately. The KL divergence in the scenario without KL annealing collapse even before the end of fast training. That's why we need a small KL coefficient in the early epochs in training. Then I tried a smoother annealing policy. In this policy, we give more early epochs on low

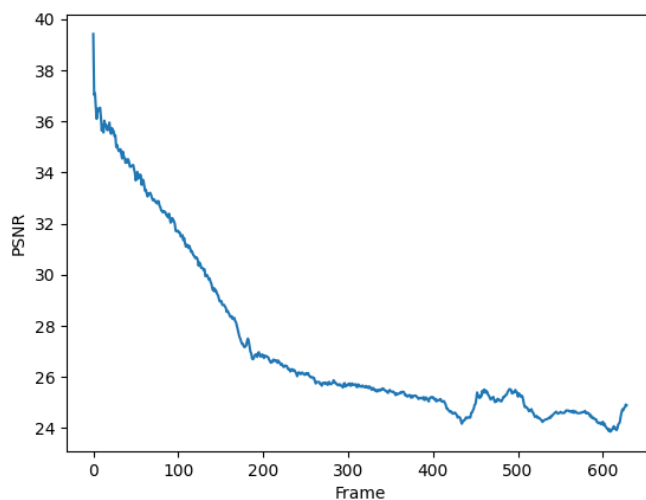


Another annealing policy

KL coefficient to mitigate the KL annealing problem.

C. Plot the PSNR-per frame diagram in validation dataset.

This is the PSNR-per frame diagram when in cyclical annealing mode after 54 epochs. Basically we have higher information about the given first frame in



PSNR-per frame

first few frames, so we can reconstruct a predicted frame with higher psnr. However, as we need to use our predicted frames to predict next frame again, the "true" info start to loss so we have a lower psnr in the late frames.

5. References

- Everybody Dance Now
- Stochastic Video Generation with a Learned Prior
- Cyclical Annealing Schedule: A Simple Approach to Mitigating KL Vanishing
- 詳解torch.nn.utils.clip_grad_norm_ 的使用與原理
- Variational Autoencoder in Pytorch
- KL divergence between two univariate Gaussians
- The reparameterization trick with code example
- 台大資訊深度學習之應用 | ADL 10.3: Variational Auto-Encoder (VAE) 控制特徵的分布以生成用
- Deep Learning 25: (1) Conditional Variational AutoEncoder : Theory (CVAE)

- The Reparameterization Trick - Clearly Explained
- Variational Inference and VAE Notes